In this chapter, you'll see:

- How to arrange your specs into meaningful groups
- How to "get the words right" with names for your groups
- Where to put shared setup and teardown code

CHAPTER 7

Structuring Code Examples

Now that you've used RSpec to design and build the beginnings of an application, you've got a mental model of "where things go." You've written short, clear examples that explain exactly what the expected behavior of the code is. You've laid these examples out in logical groups—not only to share setup code, but to keep related specs together.

By the end of this chapter, you'll be an organizational expert—for RSpec examples, at least. You'll use RSpec's flexible language to communicate your intent clearly when you organize your specs into groups. You'll also know the various places you can put shared setup code, and what the trade-offs are.

We're not asking you to memorize every possible RSpec API for code organization. We certainly haven't! But when you're deep in a big project and looking to make the tests easier to read and maintain, we hope you'll think, "Aha! I could use one of those techniques," and refer back to this chapter.

Well-structured specs are about more than just tidiness. Sometimes, you need to attach special behavior to certain examples or groups, such as setting up a database or adding custom error handling. The mechanism for this behavior—metadata—relies on good grouping, so let's talk about example groups first.

Getting the Words Right

Specs can't exist on their own. From the first spec you wrote while reading this book, every spec has been part of an example group. The example group fulfills the role of a *test case class* in other test frameworks. It has multiple purposes:

- Gives a logical structure for understanding how individual examples relate to one another